

Chapter 7

C Pointers

C How to Program, 9/e, GE

Objectives

In this chapter, you'll:

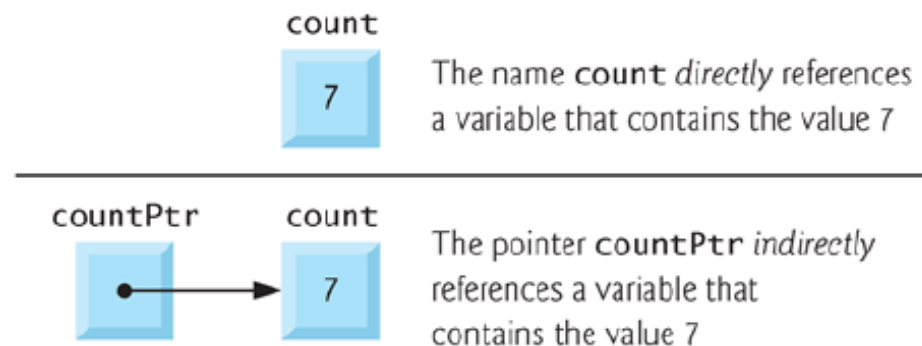
- Use pointers and pointer operators.
- Pass arguments to functions by reference using pointers.
- Understand the various placements of the `const` qualifier and how they affect what you can do with a variable.
- Use the `sizeof` operator with variables and types.
- Use pointer arithmetic to process the elements in arrays.
- Understand the close relationships among pointers, arrays and strings.
- Define and use arrays of strings.
- Use pointers to functions.
- Learn about secure C programming issues with regard to pointers.

7.1 Introduction

- Discuss one of the most powerful features of the C programming language, the **pointer**.
 - Pointers enable programs to simulate **pass-by-reference**
 - To **pass functions between functions**
 - To create and manipulate **dynamic data structures**
 - data structures that can **grow** and **shrink** at execution time
 - such as linked lists, queues, stacks and trees.
- Chapter 10 examines the **use of pointers with structures**.
- Chapter 12 introduces **dynamic memory management** techniques and presents examples of creating and using dynamic data structures.

7.2 Pointer Variable Definitions and Initialization

- Pointers are variables whose values are *memory addresses*.
 - Normally, a variable directly contains a specific *value*.
 - A pointer, on the other hand, contains an *address* of a variable that contains a specific value.
- In this sense, a variable name *directly* references a value, and a pointer *indirectly* references a value (Fig. 7.1).
- Referencing a value through a pointer is called *indirection*.



Directly and indirectly referencing a variable.

7.2 Pointer Variable Definitions and Initialization (cont.)

Declaring Pointers

- Pointers, like all variables, must be **defined** before they can be used.
- The definition

```
int *countPtr, count;
```

 - specifies that variable `countPtr`
 - is of type `int *` (i.e., a pointer to an integer) and
 - is read (right to left), “`countPtr` is a pointer to `int`” or
 - “`countPtr` points to an object of type `int`.”
- Also, the variable **`count`** is defined to be an **`int`**, *not* a pointer to an `int`.

7.2 Pointer Variable Definitions and Initialization (cont.)

Initializing and Assigning Values to Pointers

- Pointers should be **initialized** when they're defined or they can be assigned a value.
- A pointer may be initialized to **NULL, 0 or an address**.
 - A pointer with the value NULL points to *nothing*.
 - NULL is a *symbolic constant* defined in the `<stddef.h>` header (and several other headers, such as `<stdio.h>`).
- Initializing a pointer to 0 is equivalent to initializing a pointer to NULL, but **NULL is preferred**.
- When 0 is assigned, it's first converted to a pointer of the appropriate type.
- The value 0 is the **only integer value** that can be assigned directly to a pointer variable.

7.3 Pointer Operators

- The `&`, or **address operator**, is a **unary operator** that returns the **address** of its operand.
- For example, assuming the definitions

```
int y = 5;  
int *yPtr;
```

the statement

```
yPtr = &y;
```

assigns the **address** of the variable `y` to pointer variable `yPtr`.

- Variable `yPtr` is then said to “**point to**” `y`.

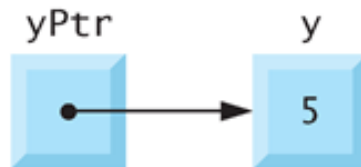


Fig. 7.2 | Graphical representation of a pointer pointing to an integer variable in memory.

7.3 Pointer Operators (Cont.)

Pointer Representation in Memory

- Figure 7.3 shows the representation of the pointer in memory, assuming that **integer variable** **y** is **stored** at location 600000, and **pointer variable** **yPtr** is **stored** at location 500000.
- The operand of the address operator must be a variable; the address operator *cannot* be applied to **constants** or **expressions**.



Fig. 7.3 | Representation of **y** and **yPtr** in memory.

7.3 Pointer Operators (Cont.)

The Indirection () Operator*

- The unary * operator
 - **indirection operator** or **dereferencing operator**
 - returns the *value* of the object to which its operand (i.e., a pointer) points.
- For example, the statement
`printf("%d", *yPtr);`
prints the value of variable `y`, namely 5.
- Using * in this manner is called **dereferencing a pointer**.



Fig. 7.3 | Representation of `y` and `yPtr` in memory.

7.3 Pointer Operators (Cont.)

*Demonstrating the & and * Operators*

- Figure 7.4 demonstrates the pointer operators & and *.
 - The printf conversion specifier **%p** outputs the memory location as a *hexadecimal integer* on most platforms.

```

1 // Fig. 7.4: fig07_04.c
2 // Using the & and * pointer operators.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     int a = 7;
8     int *aPtr = &a; // set aPtr to the address of a
9
10    printf("The address of a is %p"
11           "\nThe value of aPtr is %p", &a, aPtr);
12
13    printf("\n\nThe value of a is %d"
14           "\nThe value of *aPtr is %d", a, *aPtr);
15
16    printf("\n\nShowing that * and & are complements of "
17           "each other\n&*aPtr = %p"
18           "\n*&aPtr = %p\n", &*aPtr, *&aPtr);
19 }

```

- The address of **a** and the value of **aPtr** are identical in the output
- Confirming that the address of **a** is indeed assigned to the pointer variable **aPtr** (line 8)

- The & and * operators are **complements** of one another
- When they're both applied consecutively to **aPtr** in either order (line 21), the same result is printed.

Fig. 7.4 | Using the & and * pointer operators. (Part 1 of 2.)

```

The address of a is 0028FEC0
The value of aPtr is 0028FEC0

```

```

The value of a is 7
The value of *aPtr is 7

```

```

Showing that * and & are complements of each other
&*aPtr = 0028FEC0
*&aPtr = 0028FEC0

```

Fig. 7.4 | Using the & and * pointer operators. (Part 2 of 2.)

Operators	Associativity	Type
() [] ++ (<i>postfix</i>) -- (<i>postfix</i>)	left to right	postfix
+ - ++ -- ! * & (<i>type</i>)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Fig. 7.5 | Precedence and associativity of the operators discussed so far.

Program Demo 7_3

- Fig. 7.4

7.4 Passing Arguments to Functions by Reference

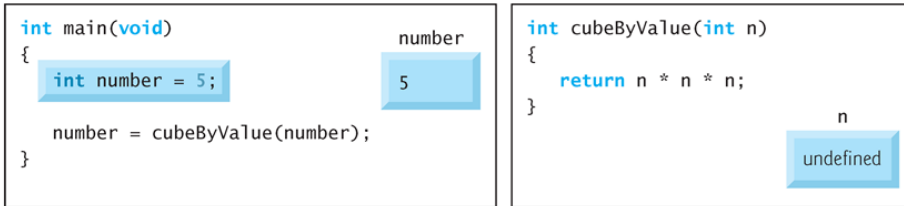
- There are two ways to pass arguments to a function
 - pass-by-value and pass-by-reference.
- *All arguments in C are **passed by value**.*
- Many functions require the capability to **modify variables in the caller** or to **pass a pointer to a large data object** to avoid the overhead of passing the object by value (which incurs the **time** and **memory overheads** of making a copy of the object).
- In C, you use **pointers** and the **indirection operator** to **simulate pass-by-reference**.

7.4 Passing Arguments to Functions by Reference (Cont.)

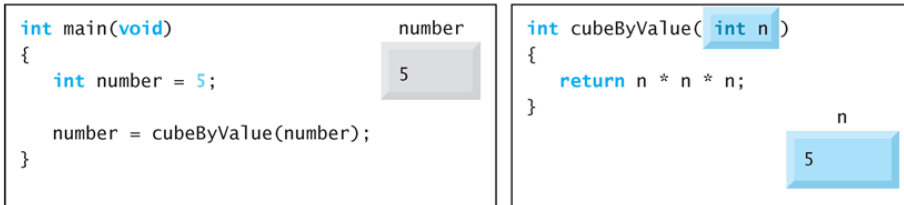
- When calling a function with arguments that **should be modified**, the **addresses of the arguments** are passed.
- This is normally accomplished by applying the address operator (&) to the variable (in the caller) whose value will be modified.
- As we saw in Chapter 6, arrays are **not** passed using operator & because C **automatically passes the starting location in memory of the array** (the name of an array is equivalent to `&arrayName[0]`).
- When the address of a variable is passed to a function, the **indirection operator (*)** may be used in the function to modify the value at that location in the caller's memory.

Pass-By-Value

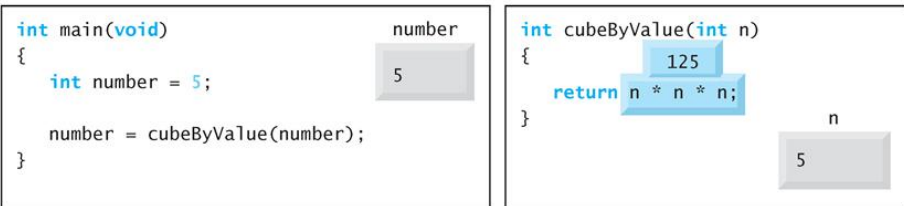
Step 1: Before main calls cubeByValue:



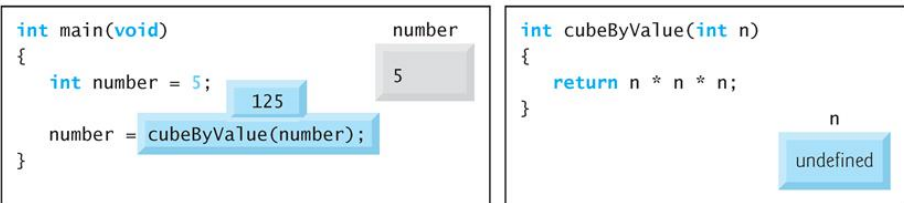
Step 2: After cubeByValue receives the call:



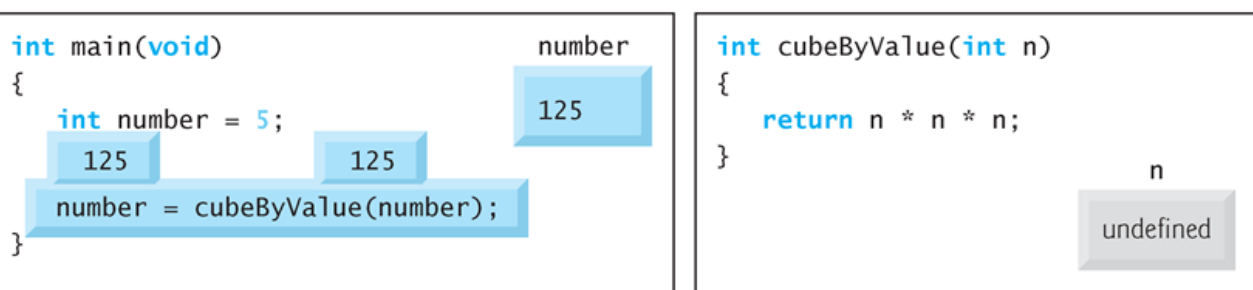
Step 3: After cubeByValue cubes parameter n and before cubeByValue returns to main:



Step 4: After cubeByValue returns to main and before assigning the result to number:



Step 5: After main completes the assignment to number:



```

1 // Fig. 7.6: fig07_06.c
2 // Cube a variable using pass-by-value.
3 #include <stdio.h>
4
5 int cubeByValue(int n); // prototype
6
7 int main(void)
8 {
9     int number = 5; // initialize number
10
11     printf("The original value of number is %d", number);
12
13     // pass number by value to cubeByValue
14     number = cubeByValue(number);
15
16     printf("\nThe new value of number is %d\n", number);
17 }
18
19 // calculate and return cube of integer argument
20 int cubeByValue(int n)
21 {
22     return n * n * n; // cube local variable n and return result
23 }

```

ig. 7.6 | Cube a variable using pass-by-value. (Part 1 of 2.)

The original value of number is 5
The new value of number is 125

Fig. 7.6 | Cube a variable using pass-by-value. (Part 2 of 2.)


```

1 // Fig. 7.7: fig07_07.c
2 // Cube a variable using pass-by-reference with a pointer argument.
3
4 #include <stdio.h>
5
6 void cubeByReference(int *nPtr); // function prototype
7
8 int main(void)
9 {
10     int number = 5; // initialize number
11
12     printf("The original value of number is %d", number);
13
14     // pass address of number to cubeByReference
15     cubeByReference(&number);
16
17     printf("\nThe new value of number is %d\n", number);
18 }
19
20 // calculate cube of *nPtr; actually modifies number in main
21 void cubeByReference(int *nPtr)
22 {
23     *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
24 }

```

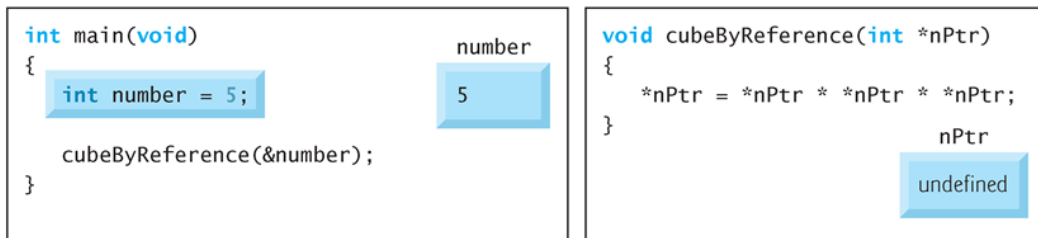
- Passes the variable **number** by **reference**
 - The **address** of **number** is passed to function **cubeByReference**.
- Function **cubeByReference** takes as a parameter a pointer to an **int** called **nPtr**.
 - The function **dereferences** the pointer and cubes the value to which **nPtr** points,
 - then assigns the result to ***nPtr** (which is really **number** in **main**), thus **changing** the value of **number** in **main**

Fig. 7.7 | Cube a variable using pass-by-reference with a pointer argument. (Part 1 of 2)

The original value of number is 5
The new value of number is 125

- The header specifies that **cubeByReference** receives the **address** of an integer variable as an argument, stores the address locally in **nPtr** and does **not** return a value.

Step 1: Before main calls cubeByReference:



Step 2: After cubeByReference receives the call and before *nPtr is cubed:

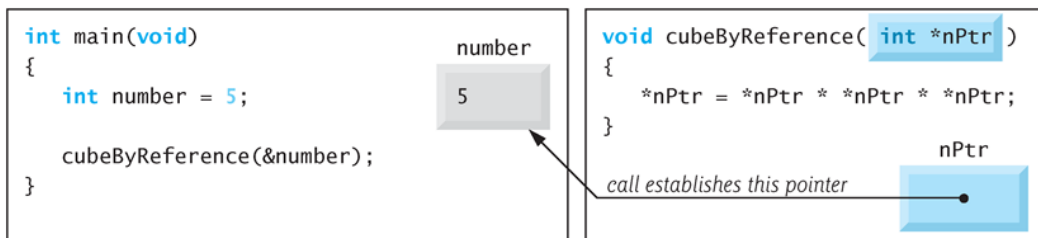


Fig. 7.9 | Analysis of a typical pass-by-reference with a pointer argument. (Part 1 of 2.)

```

1 // Fig. 7.7: fig07_07.c
2 // Cube a variable using pass-by-reference with a pointer argument.
3
4 #include <stdio.h>
5
6 void cubeByReference(int *nPtr); // function prototype
7
8 int main(void)
9 {
10     int number = 5; // initialize number
11     printf("The original value of number is %d", number);
12
13     // pass address of number to cubeByReference
14     cubeByReference(&number);
15
16     printf("\nThe new value of number is %d\n", number);
17 }
18
19 // calculate cube of *nPtr; actually modifies number in main
20 void cubeByReference(int *nPtr)
21 {
22     *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
23 }
24

```

Fig. 7.7 | Cube a variable using pass-by-reference with a pointer argument. (Part 1 of 2.)

Step 3: After *nPtr is cubed and before program control returns to main:

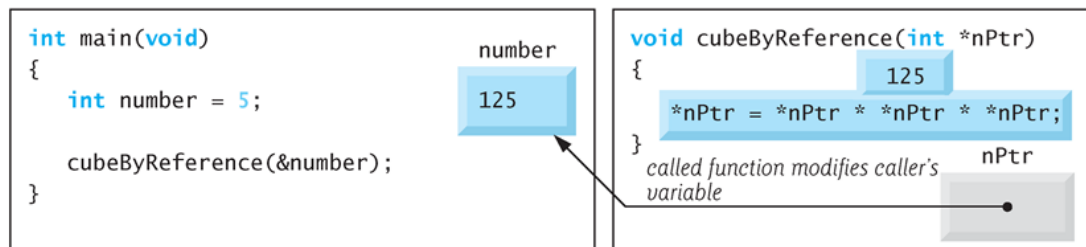


Fig. 7.9 | Analysis of a typical pass-by-reference with a pointer argument. (Part 2 of 2.)

- When the compiler encounters a function parameter for a one-dimensional array of the form **int b[]**, the compiler converts the parameter to the **pointer** notation **int *b**.
- The two forms are **interchangeable**.

Program demo

- Fig. 7.6
- Fig. 7.7

7.5 Using the `const` Qualifier with Pointers

- The `const` qualifier enables you to **inform** the **compiler** that the value of a particular variable **should not be modified**.
- If an attempt is made to modify a value that's declared **`const`**, the compiler catches it and issues either a **warning** or an **error**, depending on the particular compiler.

7.5 Using the const Qualifier with Pointers (Cont.)

- There are **four** ways to pass a pointer to a function:
 - a non-constant **pointer** to non-constant **data**,
 - a constant **pointer** to non-constant **data**,
 - a non-constant **pointer** to constant **data**, and
 - a constant **pointer** to constant **data**.
- Each of the four combinations provides **different** access privileges.

7.5.1 Converting a String to Uppercase Using a Non-Constant Pointer to Non-Constant Data

- The highest level of data access is granted by a non-constant pointer to non-constant data.
 - The data can be **modified** through the dereferenced pointer
 - The pointer can be **modified** to point to other data items.
- A declaration for a non-constant pointer to non-constant data **does not include `const`**.
- Such a pointer might be used **to receive a string** as an argument to a function that processes (and possibly modifies) each character in the string.

7.5.1 Converting a String to Uppercase Using a Non-Constant Pointer to Non-Constant Data (Cont.)

- Function `convertToUppercase` of Fig. 7.10 declares its parameter, a non-constant pointer to *non-constant data* called `sPtr` (`char *sPtr`)
- The function processes the array `string` (pointed to by `sPtr`) one character at a time.
- C standard library function `toupper` from the `<ctype.h>` header is called to convert each character to its corresponding uppercase letter—if the original character is not a letter or is already uppercase, `toupper` returns the original character.
- Line 23 moves the pointer to the next character in the string.

```

1 // Fig. 7.10: fig07_10.c
2 // Converting a string to uppercase using a
3 // non-constant pointer to non-constant data.
4 #include <stdio.h>
5 #include <ctype.h>
6
7 void convertToUppercase(char *sPtr); // prototype
8
9 int main(void)
10 {
11     char string[] = "cHaRaCters and $32.98"; // initialize char array
12
13     printf("The string before conversion is: %s", string);
14     convertToUppercase(string);
15     printf("\nThe string after conversion is: %s\n", string);
16 }
17
18 // convert string to uppercase letters
19 void convertToUppercase(char *sPtr)
20 {
21     while (*sPtr != '\0') { // current character is not '\0'
22         *sPtr = toupper(*sPtr); // convert to uppercase
23         ++sPtr; // make sPtr point to the next character
24     }
25 }

```

Function **convertToUppercase** declares its parameter, a **non-constant** pointer to **non-constant data** called **sPtr** (**char *sPtr**)

The function processes the array string (pointed to by **sPtr**) **one character** at a time.

The string before conversion is: cHaRaCters and \$32.98
The string after conversion is: CHARACTERS AND \$32.98

Fig. 7.10 | Converting a string to uppercase using a non-constant pointer to non-constant data. (Part 2 of 2.)

Line 23 **moves** the pointer to the **next character** in the string.

7.5.2 Printing a String One Character at a Time Using a Non-Constant Pointer to Constant Data

- A non-constant pointer to constant data *can be modified* to point to any data item of the appropriate type, but the *data* to which **it points cannot be modified**.
- Such a pointer might be used **to receive an array** argument to a function that will process each element **without modifying the data**.

```

1 // Fig. 7.11: fig07_11.c
2 // Printing a string one character at a time using
3 // a non-constant pointer to constant data.
4
5 #include <stdio.h>
6
7 void printCharacters(const char *sPtr);
8
9 int main(void)
10 {
11     // initialize char array
12     char string[] = "print characters of a string";
13
14     puts("The string is:");
15     printCharacters(string);
16     puts("");
17 }
18

```

```

19 // sPtr cannot be used to modify the character to which it points,
20 // i.e., sPtr is a "read-only" pointer
21 void printCharacters(const char *sPtr)
22 {
23     // loop through entire string
24     for (; *sPtr != '\0'; ++sPtr) { // no initialization
25         printf("%c", *sPtr);
26     }
27 }

```

function **printCharacters** declares parameter **sPtr** to be of type **const char ***

The declaration is read from right to left as
"sPtr is a pointer to a character constant."
The function uses a **for** statement to output
each character in the string until the **null**
character is encountered.

The string is:
print characters of a string

After each character is printed, pointer **sPtr** is incremented to point to the next character in the string.

7.5.2 Printing a String One Character at a Time Using a Non-Constant Pointer to Constant Data (Cont.)

- Figure 7.12 illustrates the attempt to compile a function that **receives a non-constant pointer (xPtr)** to constant data.
- This function attempts to modify the data pointed to by **xPtr**—**which** results in a **compilation error**.

```
1 // Fig. 7.12: fig07_12.c
2 // Attempting to modify data through a
3 // non-constant pointer to constant data.
4 #include <stdio.h>
5 void f(const int *xPtr); // prototype
6
7 int main(void)
8 {
9     int y; // define y
10
11     f(&y); // f attempts illegal modification
12 }
13
14 // xPtr cannot be used to modify the
15 // value of the variable to which it points
16 void f(const int *xPtr)
17 {
18     *xPtr = 100; // error: cannot modify a const object
19 }
```

error C2166: l-value specifies const object

Fig. 7.12 | Attempting to modify data through a non-constant pointer to constant data.

7.5.2 Printing a String One Character at a Time Using a Non-Constant Pointer to Constant Data (Cont.)

- **Arrays** are **aggregate data types** that store related data items of the **same type** under one name.
- In Chapter 10, we'll discuss another form of aggregate data type called a **structure** (sometimes called a **record** in other languages).
 - A structure is capable of storing related data items of **different data types** under one name (e.g., storing information about each employee of a company).
- When a function is called with an **array as an argument**, the **array is automatically passed to the function *by reference***.
- However, **structures** are always passed ***by value***—a ***copy*** of the entire structure is passed.

7.5.2 Printing a String One Character at a Time Using a Non-Constant Pointer to Constant Data (Cont.)

- This requires the **execution-time overhead** of making a copy of each data item in the structure and storing it on the computer's *function call stack*.
- When **structure data** must be passed to a function, we can use **pointers to constant data** to get **the performance of pass-by-reference** and **the protection of pass-by-value**.
 - When a pointer to a structure is passed, only a copy of the *address* at which the structure is stored must be made.
- On a machine with four-byte addresses, a copy of **four** bytes of memory is made rather than a copy of possibly **large** structure.

7.5.3 Attempting to Modify a Constant Pointer to Non-Constant Data

- A **constant pointer to non-constant data** always points to the **same** memory location, and the data at that location *can be modified* through the pointer.
 - This is the default for an **array name**.
 - An array name is a constant pointer to **the beginning of the array**.
 - All data in the array can be **accessed and changed** by using the array **name** and array **indexing**.
- A constant pointer to non-constant data can be used to receive an array as an argument to a function that accesses array elements **using only array index notation**.

7.5.3 Attempting to Modify a Constant Pointer to Non-Constant Data (Cont.)

- Pointers that are declared **const** must be **initialized** when they're defined
 - (if the pointer is a function parameter, it's initialized with a pointer that's passed to the function).

```

1 // Fig. 7.13: fig07_13.c
2 // Attempting to modify a constant pointer to non-constant data.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     int x; // define x
8     int y; // define y
9
10    // ptr is a constant pointer to an integer that can be modified
11    // through ptr, but ptr always points to the same memory location
12    int * const ptr = &x;
13
14    *ptr = 7; // allowed: *ptr is not const
15    ptr = &y; // error: ptr is const; cannot assign new address
16 }

```

c:\examples\ch07\fig07_13.c(15) : error C2166: l-value specifies const object

Fig. 7.13 | Attempting to modify a constant pointer to non-constant data.

- Figure 7.13 attempts to modify a constant pointer.
 - Pointer **ptr** is defined to be of type **int * const**.
 - The definition is read from *right to left* as “**ptr is a constant pointer to an integer.**”
 - The pointer is initialized with the **address** of integer variable **x**.
- The program attempts to assign the address of **y** to **ptr** (line 15), but the compiler generates an **error** message.

7.5.4 Attempting to Modify a Constant Pointer to Constant Data

- The *least access privilege* is granted by a *constant pointer to constant data*.
 - Such a pointer always points to the *same* memory location
 - The data at that memory location *cannot be modified*.
- This is how an array should be passed to a function that only looks at the array using *array index* notation and *does not modify* the array.

```

1 // Fig. 7.14: fig07_14.c
2 // Attempting to modify a constant pointer to constant data.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     int x = 5; // initialize x
8     int y; // define y
9
10    // ptr is a constant pointer to a constant integer. ptr always
11    // points to the same location; the integer at that location
12    // cannot be modified
13    const int *const ptr = &x; // initialization is OK
14
15    printf("%d\n", *ptr);
16    *ptr = 7; // error: *ptr is const; cannot assign new value
17    ptr = &y; // error: ptr is const; cannot assign new address
18 }

```

- Line 13 defines pointer variable **ptr** to be of type **const int *const**, which is read from *right to left* as “**ptr is a constant pointer to an integer constant.**”

when an attempt is made to modify the data to which **ptr** points

when an attempt is made to **modify the address** stored in the pointer variable

```

c:\examples\ch07\fig07_14.c(16) : error C2166: l-value specifies const object
c:\examples\ch07\fig07_14.c(17) : error C2166: l-value specifies const object

```

Fig. 7.14 | Attempting to modify a constant pointer to constant data.

Program Demo

- Figs. 7.10 ~7.14

7.6 Bubble Sort Using Pass-by-Reference

- Let's improve the bubble sort program of Fig. 6.15 to use two functions—`bubbleSort` and `swap`.
 - Function `bubbleSort` sorts the array.
 - It calls function `swap` to exchange the array elements `array[j]` and `array[j + 1]`
- Remember that C enforces *information hiding* between functions
 - so `swap` does **not** have access to individual array elements in `bubbleSort`.
- Because **`bubbleSort`** *wants* **`swap`** to have access to the array elements to be swapped, **`bubbleSort`** passes each of these elements *by reference* to `swap`
 - The *address* of each array element is passed explicitly.

```

1 // Fig. 6.15: fig06_15.c
2 // Sorting an array's values into ascending order.
3 #include <stdio.h>
4 #define SIZE 10
5
6 // function main begins program execution
7 int main( void )
8 {
9     // initialize a
10    int a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
11    int pass; // passes counter
12    size_t i; // comparisons counter
13    int hold; // temporary location used to swap array elements
14
15    puts( "Data items in original order" );
16
17    // output original array
18    for ( i = 0; i < SIZE; ++i ) {
19        printf( "%4d", a[ i ] );
20    } // end for
21
22    // bubble sort
23    // loop to control number of passes
24    for ( pass = 1; pass < SIZE; ++pass ) {
25
26        // loop to control number of comparisons per pass
27        for ( i = 0; i < SIZE - 1; ++i ) {
28
29            // compare adjacent elements and swap them if first
30            // element is greater than second element
31            if ( a[ i ] > a[ i + 1 ] ) {
32                hold = a[ i ];
33                a[ i ] = a[ i + 1 ];
34                a[ i + 1 ] = hold;
35            } // end if
36        } // end inner for
37    } // end outer for
38
39    puts( "\nData items in ascending order" );
40
41    // output sorted array
42    for ( i = 0; i < SIZE; ++i ) {
43        printf( "%4d", a[ i ] );
44    } // end for
45
46    puts( "" );
47 } // end main

```

- ▶ If a swap is **necessary**
 - it's performed by the **three** assignments
 - `hold = a[ i ];`
 - `a[ i ] = a[ i + 1 ];`
 - `a[ i + 1 ] = hold;`
- ▶ The swap **cannot** be performed
 - with only the **two** assignments
 - `a[ i ] = a[ i + 1 ];`
 - `a[ i + 1 ] = a[ i ];`

Data items in original order

2	6	4	8	10	12	89	68	45	37
---	---	---	---	----	----	----	----	----	----

On the **first** pass, the **largest** value is guaranteed to sink to the bottom element of the array, a[9].

2	4	6	8	10	12	68	45	37	89
---	---	---	---	----	----	----	----	----	----

only **nine** comparisons are performed

On the second pass, the second-largest value is guaranteed to sink to a[8].

2	4	6	8	10	12	45	37	68	89
---	---	---	---	----	----	----	----	----	----

On the ninth pass, the ninth-largest value sinks to a[1]. This leaves the smallest value in a[0]

Data items in ascending order

2	4	6	8	10	12	37	45	68	89
---	---	---	---	----	----	----	----	----	----

```

1 // Fig. 7.15: fig07_15.c
2 // Putting values into an array, sorting the values into
3 // ascending order and printing the resulting array.
4 #include <stdio.h>
5 #define SIZE 10
6
7 void bubbleSort( int * const array, size_t size ); // prototype
8
9 int main( void )
10 {
11     // initialize array a
12     int a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
13
14     size_t i; // counter
15
16     puts( "Data items in original order" );
17
18     // loop through array a
19     for ( i = 0; i < SIZE; ++i ) {
20         printf( "%4d", a[ i ] );
21     } // end for
22
23     bubbleSort( a, SIZE ); // sort the array
24
25     puts( "\nData items in ascending order" );
26
27     // loop through array a
28     for ( i = 0; i < SIZE; ++i ) {
29         printf( "%4d", a[ i ] );
30     } // end for
31
32     puts( "" );
33 } // end main
34
35 // sort an array of integers using bubble sort algorithm
36 void bubbleSort( int * const array, size_t size )
37 {
38     void swap( int *element1Ptr, int *element2Ptr ); // prototype
39     unsigned int pass; // pass counter
40     size_t j; // comparison counter
41
42     // loop to control passes
43     for ( pass = 0; pass < size - 1; ++pass ) {
44
45         // loop to control comparisons during each pass
46         for ( j = 0; j < size - 1; ++j ) {
47
48             // swap adjacent elements if they're out of order
49             if ( array[ j ] > array[ j + 1 ] ) {
50                 swap( &array[ j ], &array[ j + 1 ] );
51             } // end if
52         } // end inner for
53     } // end outer for
54 } // end function bubbleSort
55
56 // swap values at memory locations to which element1Ptr and
57 // element2Ptr point
58 void swap( int *element1Ptr, int *element2Ptr )
59 {
60     int hold = *element1Ptr;
61     *element1Ptr = *element2Ptr;
62     *element2Ptr = hold;
63 } // end function swap

```

Data items in original order	2	6	4	8	10	12	89	68	45	37
Data items in ascending order	2	4	6	8	10	12	37	45	68	89

Arrays are automatically passed by **reference**
 individual array elements are scalars and are ordinarily passed by **value**.
 Uses the **address** operator (&) on each of the array elements in the swap call to effect **pass-by-reference**

swap—because of information hiding—is **not** allowed to know the name array[j]
 swap may use *element1Ptr as a **synonym** for array[j]
 when swap references *element1Ptr, it's *actually* referencing array[j] in bubbleSort.
 when swap references *element2Ptr, it's *actually* referencing array[j + 1] in bubbleSort.

Program Demo

- Fig. 7_15

7.7 sizeof Operator

- C provides the special unary operator `sizeof` to **determine the size in bytes of an array** (or any other data type) .


```

1 // Fig. 7.16: fig07_16.c
2 // Applying sizeof to an array name returns
3 // the number of bytes in the array.
4 #include <stdio.h>
5 #define SIZE 20
6
7 size_t getSize(float *ptr); // prototype
8
9 int main(void)
10 {
11     float array[SIZE]; // create array
12
13     printf("The number of bytes in the array is %u"
14           "\nThe number of bytes returned by getSize is %u\n",
15           sizeof(array), getSize(array));
16 }
17
18 // return size of ptr
19 size_t getSize(float *ptr)
20 {
21     return sizeof(ptr);
22 }

```

- When applied to the **name** of an array as in Fig. 7.16, the **sizeof** operator returns the **total number of bytes** in the array as type **size_t**.

The number of bytes in the array is 80
The number of bytes returned by getSize is 4

Fig. 7.16 | Applying sizeof to an array name returns the number of bytes in the array. (Part 2 of 2)

- Variables of type **float** on this computer are stored in **4** bytes of memory, and array is defined to have 20 elements.
- Therefore, there are a total of **80** bytes in array

7.7 sizeof Operator (Cont.)

- The number of elements in an array also can be determined with `sizeof`.
- For example, consider the following array definition:
`double real[22];`
- Variables of type `double` normally are stored in **8 bytes** of memory.
- Thus, array `real` contains a total of **176** bytes.
- To determine **the number of elements** in the array, the following expression can be used:

`sizeof(real) / sizeof(real[0])`

7.7 sizeof Operator (Cont.)

Determining the Sizes of the Standard Types, an Array and a Pointer

- Figure 7.17 calculates the **number of bytes** used to store each of the standard data types.
- *The results of this program are **implementation dependent** and often **differ across platforms** and sometimes across different compilers on the same platform.*

```

1 // Fig. 7.17: fig07_17.c
2 // Using operator sizeof to determine standard data type sizes.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     char c;
8     short s;
9     int i;
10    long l;
11    long long ll;
12    float f;
13    double d;
14    long double ld;
15    int array[20]; // create array of 20 int elements
16    int *ptr = array; // create pointer to array
17
18    printf("    sizeof c = %u\\tsizeof(char) = %u"
19           "\\n    sizeof s = %u\\tsizeof(short) = %u"
20           "\\n    sizeof i = %u\\tsizeof(int) = %u"
21           "\\n    sizeof l = %u\\tsizeof(long) = %u"
22           "\\n    sizeof ll = %u\\tsizeof(long long) = %u"
23           "\\n    sizeof f = %u\\tsizeof(float) = %u"
24           "\\n    sizeof d = %u\\tsizeof(double) = %u"
25           "\\n    sizeof ld = %u\\tsizeof(long double) = %u"
26           "\\n    sizeof array = %u"
27           "\\n    sizeof ptr = %u\\n",
28           sizeof c, sizeof(char), sizeof s, sizeof(short), sizeof i,
29           sizeof(int), sizeof l, sizeof(long), sizeof ll,
30           sizeof(long long), sizeof f, sizeof(float), sizeof d,
31           sizeof(double), sizeof ld, sizeof(long double),
32           sizeof array, sizeof ptr);
33 }

```

sizeof c = 1	sizeof(char) = 1
sizeof s = 2	sizeof(short) = 2
sizeof i = 4	sizeof(int) = 4
sizeof l = 4	sizeof(long) = 4
sizeof ll = 8	sizeof(long long) = 8
sizeof f = 4	sizeof(float) = 4
sizeof d = 8	sizeof(double) = 8
sizeof ld = 8	sizeof(long double) = 8
sizeof array = 80	
sizeof ptr = 4	

Program Demo

- Figs. 7.16~7.17

7.8 Pointer Expressions and Pointer Arithmetic

- Pointers are **valid** operands in **arithmetic** expressions, **assignment** expressions and **comparison** expressions.
- However, **not** all the operators normally used in these expressions are valid in conjunction with pointer variables.
- Describes the operators that can have pointers as **operands**, and how these operators are used.
- A limited set of **arithmetic** operations may be performed on pointers.

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- A pointer may be *incremented* (++) or *decremented* (--),
 - an integer may be **added** to a pointer (+ or +=),
 - an integer may be **subtracted** from a pointer (- or -=)
 - one pointer may be subtracted from another
 - this last operation is meaningful only when **both** pointers point to elements of the **same** array.

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- Assume that array `int v[5]` has been defined and its first element is at location 3000 in memory.
- Assume pointer `vPtr` has been initialized to point to `v[0]`—i.e., the value of `vPtr` is 3000.
- Figure 7.18 illustrates this situation for a machine with 4-byte integers.
- Variable `vPtr` can be initialized to point to array `v` with either of the statements

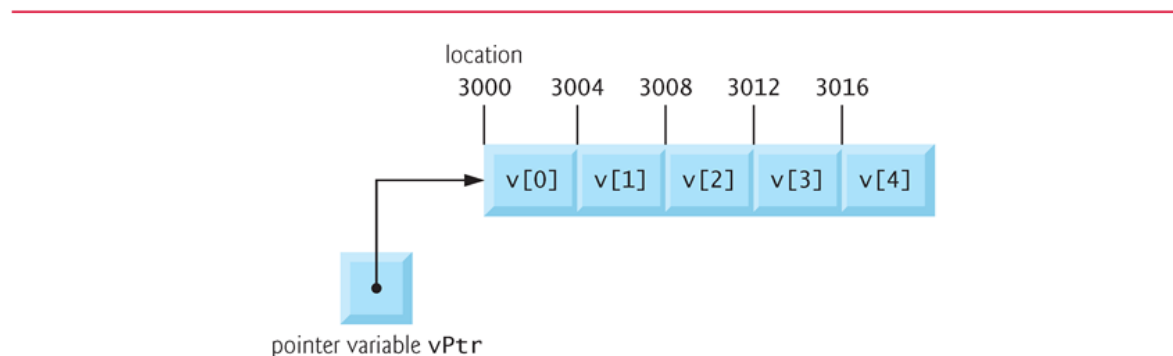


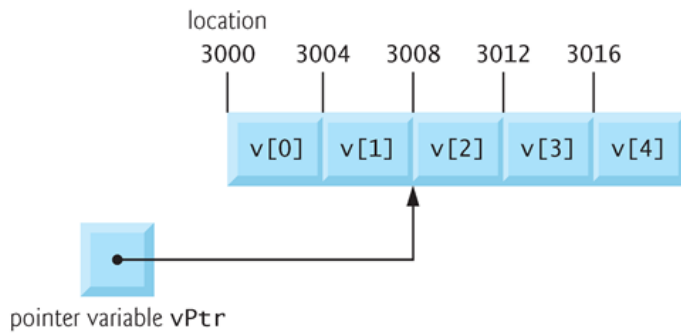
Fig. 7.18 | Array `v` and a pointer variable `vPtr` that points to `v`.

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- In conventional **arithmetic**, $3000 + 2$ yields the value 3002.
- This is normally **not** the case with pointer arithmetic.
- When an integer is added to or subtracted from a pointer
 - The pointer is **not** incremented or decremented simply by that integer
 - But by that **integer times the size of the object** to which the pointer refers.
- The **number of bytes** depends on the **object's data type**.

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- For example, the statement
`vPtr += 2;`
would produce 3008 ($3000 + 2 * 4$), assuming an integer is stored in 4 bytes of memory.
- In the array `v`, `vPtr` would now point to `v[2]`.



Either of the statements `++vPtr;` `vPtr++;` increments the pointer to point to the next location in the array.

Either of the statements `--vPtr;` `vPtr--;` decrements the pointer to point to the previous element of the array.

Fig. 7.19 | The pointer `vPtr` after pointer arithmetic.

If `vPtr` had been incremented to 3016, which points to `v[4]`, the statement `vPtr -= 4;` would set `vPtr` back to 3000—the beginning of the array.

- If an integer is stored in 2 bytes of memory, then the preceding calculation would result in memory location 3004 ($3000 + 2 * 2$).

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer variables may be **subtracted** from one another.
- For example, if **vPtr** contains the location 3000, and **v2Ptr** contains the address 3008, the statement

$x = v2Ptr - vPtr;$

would assign to **x** *the number of array elements* from **vPtr** to **v2Ptr**, in this case **2** (not 8).

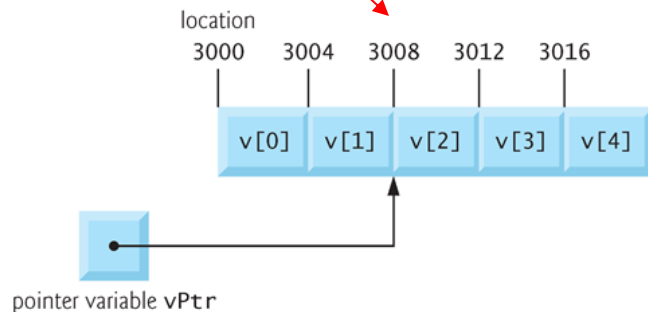


Fig. 7.19 | The pointer **vPtr** after pointer arithmetic.

- Pointer arithmetic is **undefined** unless performed on an array.
- We **cannot** assume that **two variables of the same type** are stored contiguously in memory **unless** they're **adjacent elements** of an array.

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- A pointer **can be assigned** to another pointer if both have the **same type**.
- The exception to this rule is the **pointer to void** (i.e., **void ***), which is a generic pointer that can represent **any** pointer type.
- All pointer types can be assigned a pointer to **void**, and a pointer to **void** can be assigned a pointer of any type.
- In both cases, a **cast operation is not required**.
- A pointer to void **cannot** be dereferenced.

7.8 Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointers can be compared using **equality** and **relational operators**, but such comparisons are **meaningless** unless the pointers **point to elements of the *same* array**.
- Pointer comparisons compare the **addresses** stored in the pointers.

7.9 Relationship between Pointers and Arrays

- Arrays and pointers are intimately (密切地) related in C and often may be used interchangeably.
 - An *array name* can be thought of as a *constant pointer*.
 - **Pointers** can be used to do any operation involving **array indexing**.
- Assume that integer array `b[5]` and integer pointer variable `bPtr` have been defined.
 - Because the **array name** (without an index) is a **pointer to the first element of the array**, we can set `bPtr` equal to the **address** of the first element in array `b` with the statement `bPtr = b;`

7.9 Relationship between Pointers and Arrays (Cont.)

- This statement is equivalent to taking the **address** of the array's first element as follows:

`bPtr = &b[0];` $\longleftrightarrow$ `bPtr = b;`

- Array element `b[3]` can alternatively be referenced with the pointer expression

`*(bPtr + 3)`

- The 3 in the expression is the **offset** to the pointer.
- When the pointer points to the array's first element, the offset indicates which array element should be referenced, and the **offset value is identical to the array index**.
- This notation is referred to as **pointer/offset notation**.

7.9 Relationship between Pointers and Arrays (Cont.)

- The **parentheses are necessary** because the precedence of `*` is **higher** than the precedence of `+`.
 - Without the parentheses, the above expression would add 3 to the value of the expression `*bPtr` (i.e., **3 would be added to `b[0]`**, assuming `bPtr` points to the beginning of the array).
- Just as the **array element can be referenced** with a pointer expression, the address
$$\&b[3] \iff bPtr + 3$$
can be written with the **pointer expression**

7.9 Relationship between Pointers and Arrays (Cont.)

- The **array itself can be treated as a pointer** and used in pointer arithmetic.
 - For example, the expression
 $*(b + 3)$
also refers to the array element `b[3]`.
- In general, all **indexed array expressions** can be written with **a pointer and an offset**.
- In this case, **pointer/offset notation** was used with **the name of the array** as a **pointer**.
- The preceding statement does not modify the array name in any way; **b still points to the first element in the array**.

7.9 Relationship between Pointers and Arrays (Cont.)

- Pointers can be **indexed** like arrays.
 - If bPtr has the value b, the expression
bPtr[**1**]
refers to the array element b[1].
- This is referred to as **pointer/index notation**.
- Remember that an **array name** is essentially a constant pointer
 - It always **points to the beginning of the array**.
- Thus, the expression
b += **3**
is *invalid* because it attempts to modify the value of the array name with pointer arithmetic.

7.9 Relationship between Pointers and Arrays (Cont.)

- Figure 7.20 uses the **four** methods we've discussed for referring to array elements
 - array indexing,
 - pointer/offset with the array name as a pointer,
 - pointer indexing, and
 - pointer/offset with a pointer
- To print the four elements of the integer array b.

```

1 // Fig. 7.20: fig07_20.cpp
2 // Using indexing and pointer notations with arrays.
3 #include <stdio.h>
4 #define ARRAY_SIZE 4
5
6 int main(void)
7 {
8     int b[] = {10, 20, 30, 40}; // create and initialize array b
9     int *bPtr = b; // create bPtr and point it to array b
10
11     // output array b using array index notation
12     puts("Array b printed with:\nArray index notation");
13
14     // loop through array b
15     for (size_t i = 0; i < ARRAY_SIZE; ++i) {
16         printf("b[%u] = %d\n", i, b[i]);
17     }
18
19     // output array b using array name and pointer/offset notation
20     puts("\nPointer/offset notation where\n"
21         "the pointer is the array name");
22
23     // loop through array b
24     for (size_t offset = 0; offset < ARRAY_SIZE; ++offset) {
25         printf("(b + %u) = %d\n", offset, *(b + offset));
26     }
27
28     // output array b using bPtr and array index notation
29     puts("\nPointer index notation");
30
31     // loop through array b
32     for (size_t i = 0; i < ARRAY_SIZE; ++i) {
33         printf("bPtr[%u] = %d\n", i, bPtr[i]);
34     }
35
36     // output array b using bPtr and pointer/offset notation
37     puts("\nPointer/offset notation");
38
39     // loop through array b
40     for (size_t offset = 0; offset < ARRAY_SIZE; ++offset) {
41         printf("(bPtr + %u) = %d\n", offset, *(bPtr + offset));
42     }
43 }

```

Array b printed with:
 Array index notation
 b[0] = 10
 b[1] = 20
 b[2] = 30
 b[3] = 40

Pointer/offset notation where
 the pointer is the array name
 *(b + 0) = 10
 *(b + 1) = 20
 *(b + 2) = 30
 *(b + 3) = 40

Pointer index notation
 bPtr[0] = 10
 bPtr[1] = 20
 bPtr[2] = 30
 bPtr[3] = 40

Pointer/offset notation
 *(bPtr + 0) = 10
 *(bPtr + 1) = 20
 *(bPtr + 2) = 30
 *(bPtr + 3) = 40

Fig. 7.20 | Using indexing and pointer

7.9.5 Relationship between Pointers and Arrays (Cont.)

String Copying with Arrays and Pointers

- To further illustrate the **interchangeability** of arrays and pointers, let's look at the **two string-copying functions**—`copy1` and `copy2`—in the program of Fig. 7.21.
 - Both functions copy **a string** into **a character array**.
 - After a comparison of the function prototypes for `copy1` and `copy2`, the functions appear identical.
 - They accomplish the **same task**; but they're **implemented differently**.

```

1 // Fig. 7.21: fig07_21.c
2 // Copying a string using array notation and pointer notation.
3 #include <stdio.h>
4 #define SIZE 10
5
6 void copy1(char * const s1, const char * const s2); // prototype
7 void copy2(char *s1, const char *s2); // prototype
8
9 int main(void)
10 {
11     char string1[SIZE]; // create array string1
12     char *string2 = "Hello"; // create a pointer to a string
13
14     copy1(string1, string2);
15     printf("string1 = %s\n", string1);
16
17     char string3[SIZE]; // create array string3
18     char string4[] = "Good Bye"; // create an array containing a string
19
20     copy2(string3, string4);
21     printf("string3 = %s\n", string3);
22 }
23
24 // copy s2 to s1 using array notation
25 void copy1(char * const s1, const char * const s2)
26 {
27     // loop through strings
28     for (size_t i = 0; (s1[i] = s2[i]) != '\0'; ++i) {
29         ; // do nothing in body
30     }
31 }
32
33 // copy s2 to s1 using pointer notation
34 void copy2(char *s1, const char *s2)
35 {
36     // loop through strings
37     for (; (*s1 = *s2) != '\0'; ++s1, ++s2) {
38         ; // do nothing in body
39     }
40 }

```

```

string1 = Hello
string3 = Good Bye

```

- Function `copy1` uses **array index** notation to copy the string in `s2` to the character array `s1`.
 - The function defines counter variable `i` as the array **index**.
 - The **for** statement header performs the entire copy operation—its body is the empty statement.
 - The header specifies that `i` is initialized to zero and incremented by one on each iteration of the loop.
 - The expression `s1[i] = s2[i]` **copies** one character from `s2` to `s1`.
 - When the **null** character is encountered in `s2`, it's assigned to `s1`, and the value of the assignment becomes the value assigned to the left operand (`s1`).
 - The loop terminates when the **null** character is assigned from `s1` to `s2` (**false**).

```

1 // Fig. 7.21: fig07_21.c
2 // Copying a string using array notation and pointer notation.
3 #include <stdio.h>
4 #define SIZE 10
5
6 void copy1(char * const s1, const char * const s2); // prototype
7 void copy2(char *s1, const char *s2); // prototype
8
9 int main(void)
10 {
11     char string1[SIZE]; // create array string1
12     char *string2 = "Hello"; // create a pointer to a string
13
14     copy1(string1, string2);
15     printf("string1 = %s\n", string1);
16
17     char string3[SIZE]; // create array string3
18     char string4[] = "Good Bye"; // create an array containing a string
19
20     copy2(string3, string4);
21     printf("string3 = %s\n", string3);
22 }
23
24 // copy s2 to s1 using array notation
25 void copy1(char * const s1, const char * const s2)
26 {
27     // loop through strings
28     for (size_t i = 0; (s1[i] = s2[i]) != '\0'; ++i) {
29         ; // do nothing in body
30     }
31 }
32
33 // copy s2 to s1 using pointer notation
34 void copy2(char *s1, const char *s2)
35 {
36     // loop through strings
37     for (; (*s1 = *s2) != '\0'; ++s1, ++s2) {
38         ; // do nothing in body
39     }
40 }

```

```

string1 = Hello
string3 = Good Bye

```

Function `copy2` uses *pointers and pointer arithmetic* to copy the string in `s2` to the character array `s1`.

- Again, the for statement header performs the entire copy operation.
- The header does not include any variable initialization.
- As in function `copy1`, the expression `(*s1 = *s2)` performs the copy operation.
- Pointer `s2` is **dereferenced**, and the resulting character is assigned to the **dereferenced** pointer `*s1`.
- After the assignment in the condition, the pointers are **incremented** to point to the next element of array `s1` and the next character of string `s2`, respectively.
- When the **null** character is encountered in `s2`, it's assigned to the **dereferenced** pointer `s1` and the loop **terminates**.

Program Demo

- Figs. 7-20 ~7-21

7.10 Arrays of Pointers

- Arrays may contain pointers.
- A common use of an **array of pointers** is to form an **array of strings**
 - referred to simply as a **string array**.
 - Each entry in the array is a string, but in C a string is essentially **a pointer to its first character**.
- So each entry in an array of strings is actually a pointer to **the first character of a string**.
- Consider the definition of string array `suit`, which might be useful in representing a deck of cards.

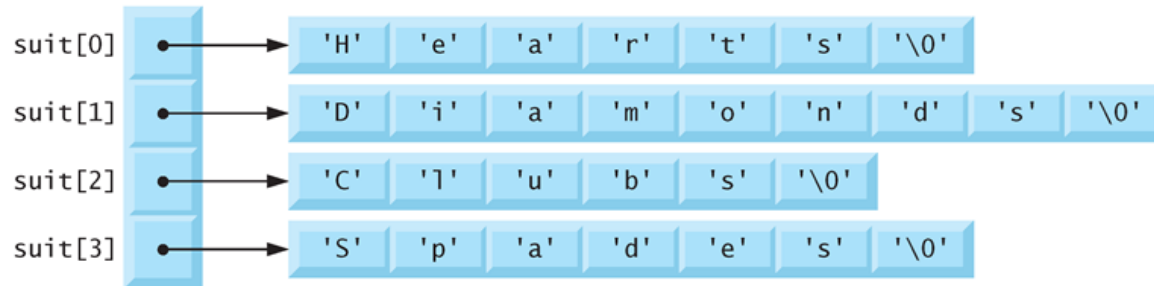
```
const char *suit[4] = {"Hearts", "Diamonds",  
"Clubs", "Spades"};
```

7.10 Arrays of Pointers (Cont.)

- The `suit[4]` portion of the definition indicates an array of 4 elements.
- The `char *` portion of the declaration indicates that each element of array `suit` is of type “**pointer to char.**”
- Qualifier `const` indicates that the strings pointed to by each element pointer will **not be modified**.
- The four values to be placed in the array are "Hearts", "Diamonds", "Clubs" and "Spades".
- Each is stored in memory as a *null-terminated character string* that's one character longer than the number of characters between quotes.

7.10 Arrays of Pointers (Cont.)

- The four strings are 7, 9, 6 and 7 characters long, respectively.
- Although it appears as though these strings are being placed in the `suit` array, only **pointers** are actually stored in the array (Fig. 7.22).
 - Each pointer points to the first character of its corresponding string.

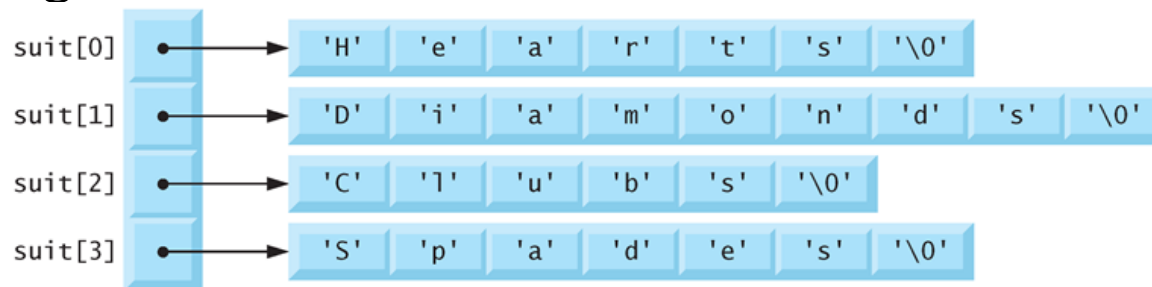


7.22 | Graphical representation of the `suit` array.

- Thus, even though the `suit` array is **fixed** in size, it provides access to character strings of **any length**.
- This flexibility is one example of C's powerful data-structuring capabilities.

7.10 Arrays of Pointers (Cont.)

- The suits could have been placed in a **two-dimensional array**, in which each **row** would represent a suit and each **column** would represent a letter from a suit name.
- Such a data structure would have to have **a fixed number of columns per row**, and that number would have to be as large as the **largest** string.
- Therefore, considerable memory **could be wasted** when storing a large number of strings of which most were shorter than the longest string.



7.11 Case Study: Card Shuffling and Dealing Simulation

- Use **random number** generation to develop a **card shuffling** and **dealing simulation** program.
 - This program can then be used to implement programs that **play specific card games**.
 - To reveal some subtle performance problems, we've intentionally used **suboptimal** shuffling and dealing algorithms.
- In this chapter's exercises and in Chapter 10, we develop more efficient algorithms.
- Using the top-down, stepwise refinement approach, we develop a program that will **shuffle** a deck of 52 playing cards and then **deal** each of the 52 cards.

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- The top-down approach is particularly useful in **attacking larger, more complex problems** than you've seen in earlier chapters.
- We use 4-by-13 two-dimensional array **deck** to represent the deck of playing cards (Fig. 7.23).

		Ace	Two	Three	Four	Five	Six	Seven	Eight	Nine	Ten	Jack	Queen	King
		0	1	2	3	4	5	6	7	8	9	10	11	12
Hearts	0													
Diamonds	1													
Clubs	2													
Spades	3													

deck[2][12] represents the King of Clubs

Clubs King

- The **rows correspond to the suits**—row 0 corresponds to hearts, row 1 to diamonds, row 2 to clubs and row 3 to spades.
- The **columns correspond to the face values of the cards**—0 through 9 correspond to ace through ten, and columns 10 through 12 correspond to jack, queen and king.
- We shall load **string array suit** with character strings representing the four suits, and **string array face** with character strings representing the thirteen face values.

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- This simulated deck of cards may be *shuffled* as follows.
 - First the array `deck` is cleared to **zeros**.
 - Then, a row (0–3) and a column (0–12) are each chosen *at random*.
 - The number 1 is inserted in array element `deck[row][column]` to indicate that this card will be the **first** one dealt from the shuffled deck.
- This process continues with the numbers 2, 3, ..., 52 being randomly inserted in the **deck** array to indicate which cards are to be placed second, third, ..., and fifty-second in the shuffled deck.

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- As the deck array begins to fill with card numbers, it's possible that a card will be selected again—i.e., `deck[row][column]` will be **nonzero** when it's selected.
 - This selection is simply **ignored** and other rows and columns are repeatedly chosen at random until an ***unselected card is found***.
 - Eventually, the numbers 1 through 52 will occupy the 52 slots of the **deck** array.
- At this point, the deck of cards is fully shuffled.

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- This shuffling algorithm can execute *indefinitely* if cards that have **already** been shuffled are repeatedly selected at random.
- This phenomenon is known as *indefinite postponement*.
- In this chapter's exercises, we discuss a **better shuffling** algorithm that **eliminates** the possibility of indefinite postponement.

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- To deal the first card, we search the array for `deck[row][column]` equal to 1.
 - This is accomplished with nested `for` statements that vary `row` from 0 to 3 and `column` from 0 to 12.
- What card does that element of the array correspond to?
 - The `suit` array has been preloaded with the four suits, so to get the suit, we print the character string `suit[row]`.
 - Similarly, to get the face value of the card, we print the character string `face[column]`.
 - We also print the character string " of ".

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- Printing this information in the proper order enables us to print each card in the form "King of Clubs", "Ace of Diamonds" and so on.
- Let's proceed with the top-down, stepwise refinement process.
- The *top* is simply
 - *Shuffle and deal 52 cards*
- Our *first* refinement yields:
 - *Initialize the suit array*
Initialize the face array
Initialize the deck array
Shuffle the deck
Deal 52 cards

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- “Shuffle the deck” may be expanded as follows:
 - *For each of the 52 cards*
Place card number in randomly selected unoccupied slot of deck
- “Deal 52 cards” may be expanded as follows:
 - *For each of the 52 cards*
Find card number in deck array and print face and suit of card

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- Incorporating these expansions yields our complete *second refinement*:
 - *Initialize the suit array*
Initialize the face array
Initialize the deck array

For each of the 52 cards
Place card number in randomly selected
unoccupied slot of deck

For each of the 52 cards
Find card number in deck array and print face and
suit of card

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- “Place card number in randomly selected unoccupied slot of deck” may be expanded as:
 - *Choose slot of deck randomly*

*While chosen slot of deck has been previously chosen
Choose slot of deck randomly*

Place card number in chosen slot of deck

- “Find card number in deck array and print face and suit of card” may be expanded as:
 - *For each slot of the deck array*
If slot contains card number
Print the face and suit of the card

7.11 Case Study: Card Shuffling and Dealing Simulation (Cont.)

- Incorporating these expansions yields our *third refinement*:

- *Initialize the suit array*
Initialize the face array
Initialize the deck array

- For each of the 52 cards*
Choose slot of deck randomly

- While slot of deck has been previously chosen*
Choose slot of deck randomly

- Place card number in chosen slot of deck*

- For each of the 52 cards*
For each slot of deck array
If slot contains desired card number
Print the face and suit of the card

```

1 // Fig. 7.24: fig07_24.c
2 // Card shuffling and dealing.
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6
7 #define SUITS 4
8 #define FACES 13
9 #define CARDS 52
10
11 // prototypes
12 void shuffle( unsigned int wDeck[][ FACES ] ); // shuffling modifies wDeck
13 void deal( unsigned int wDeck[][ FACES ], const char *wFace[],
14           const char *wSuit[] ); // dealing doesn't modify the arrays
15
16 int main( void )
17 {
18     // initialize suit array
19     const char *suit[ SUITS ] =
20     { "Hearts", "Diamonds", "Clubs", "Spades" };
21
22     // initialize face array
23     const char *face[ FACES ] =
24     { "Ace", "Deuce", "Three", "Four",
25       "Five", "Six", "Seven", "Eight",
26       "Nine", "Ten", "Jack", "Queen", "King" };
27
28     // initialize deck array
29     unsigned int deck[ SUITS ][ FACES ] = { 0 };
30
31     srand( time( NULL ) ); // seed random-number generator
32
33     shuffle( deck ); // shuffle the deck
34     deal( deck, face, suit ); // deal the deck
35 } // end main
36
37 // shuffle cards in deck
38 void shuffle( unsigned int wDeck[][ FACES ] )
39 {
40     size_t row; // row number
41     size_t column; // column number
42     size_t card; // counter
43
44     // for each of the cards, choose slot of deck randomly
45     for ( card = 1; card <= CARDS; ++card ) {
46
47         // choose new random location until unoccupied slot found
48         do {
49             row = rand() % SUITS;
50             column = rand() % FACES;
51         } while( wDeck[ row ][ column ] != 0 ); // end do...while
52
53         // place card number in chosen slot of deck
54         wDeck[ row ][ column ] = card;
55     } // end for
56 } // end function shuffle
57

```

```

58 // deal cards in deck
59 void deal( unsigned int wDeck[][ FACES ], const char *wFace[],
60           const char *wSuit[] )
61 {
62     size_t card; // card counter
63     size_t row; // row counter
64     size_t column; // column counter
65
66     // deal each of the cards
67     for ( card = 1; card <= CARDS; ++card ) {
68         // loop through rows of wDeck
69         for ( row = 0; row < SUITS; ++row ) {
70             // loop through columns of wDeck for current row
71             for ( column = 0; column < FACES; ++column ) {
72                 // if slot contains current card, display card
73                 if ( wDeck[ row ][ column ] == card ) {
74                     printf( "%5s of %-8s%c", wFace[ column ], wSuit[ row ],
75                           card % 2 == 0 ? '\n' : '\t' ); // 2-column format
76                 } // end if
77             } // end for
78         } // end for
79     } // end for
80 } // end function deal

```

Nine of Hearts	Five of Clubs
Queen of Spades	Three of Spades
Queen of Hearts	Ace of Clubs
King of Hearts	Six of Spades
Jack of Diamonds	Five of Spades
Seven of Hearts	King of Clubs
Three of Clubs	Eight of Hearts
Three of Diamonds	Four of Diamonds
Queen of Diamonds	Five of Diamonds
Six of Diamonds	Five of Hearts
Ace of Spades	Six of Hearts
Nine of Diamonds	Queen of Clubs
Eight of Spades	Nine of Clubs
Deuce of Clubs	Six of Clubs
Deuce of Spades	Jack of Clubs
Four of Clubs	Eight of Clubs
Four of Spades	Seven of Spades
Seven of Diamonds	Seven of Clubs
King of Spades	Ten of Diamonds
Jack of Hearts	Ace of Hearts
Jack of Spades	Ten of Clubs
Eight of Diamonds	Deuce of Diamonds
Ace of Diamonds	Nine of Spades
Four of Hearts	Deuce of Hearts
King of Diamonds	Ten of Spades
Three of Hearts	Ten of Hearts

There's a **weakness** in the dealing algorithm. Once a match is found, the two inner **for** statements **continue** searching the remaining elements of deck for a **match**.

Program Demo

- Fig. 7-24

7.12 Pointers to Functions

- A **pointer to a function** contains the address of the function in memory.
- In Chapter 6, we saw that an array name is really the address in memory of the first element of the array.
- Similarly, a **function name** is really the **starting address in memory of the code** that performs the function's task.
- Pointers to functions can be passed to functions, *returned* from functions, stored in arrays and *assigned* to other function pointers.

7.12 Pointers to Functions (Cont.)

- To illustrate the use of **pointers to functions**, Fig. 7.26 presents a modified version of the bubble sort program in Fig. 7.15.
 - The new version consists of `main` and functions `bubble`, `swap`, `ascending` and `descending`.
 - Function `bubbleSort` receives a **pointer to a function**
 - either function `ascending` or function `descending`
 - as an *argument*, in addition to **an integer array** and **the size of the array**.

7.12 Pointers to Functions (Cont.)

- The program prompts the user to choose whether the array should be sorted in *ascending* or in *descending* order.
 - If the user enters 1, a pointer to function *ascending* is passed to function `bubble`, causing the array to be sorted into *increasing* order.
 - If the user enters 2, a pointer to function *descending* is passed to function `bubble`, causing the array to be sorted into *decreasing* order.
- The output of the program is shown in Fig. 7.27.

```
1 // Fig. 7.26: fig07_26.c
2 // Multipurpose sorting program using function pointers.
3 #include <stdio.h>
4 #define SIZE 10
5
6 // prototypes
7 void bubble(int work[], size_t size, int (*compare)(int a, int b) );
8 int ascending(int a, int b);
9 int descending(int a, int b);
10
11 int main(void)
12 {
13     // initialize unordered array a
14     int a[SIZE] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
15
16     printf("%s", "Enter 1 to sort in ascending order,\n"
17             "Enter 2 to sort in descending order: ");
18     int order; // 1 for ascending order or 2 for descending order
19     scanf("%d", &order);
20
21     puts("\nData items in original order");
22
```

Fig. 7.26 | Multipurpose sorting program using function pointers. (Part I of 4.)

```
23 // output original array
24 for (size_t counter = 0; counter < SIZE; ++counter) {
25     printf("%5d", a[counter]);
26 }
27
28 // sort array in ascending order; pass function ascending as an
29 // argument to specify ascending sorting order
30 if (order == 1) {
31     bubble(a, SIZE, ascending);
32     puts("\nData items in ascending order");
33 }
34 else { // pass function descending
35     bubble(a, SIZE, descending);
36     puts("\nData items in descending order");
37 }
38
39 // output sorted array
40 for (size_t counter = 0; counter < SIZE; ++counter) {
41     printf("%5d", a[counter]);
42 }
43
44 puts("\n");
45 }
```

Fig. 7.26 | Multipurpose sorting program using function pointers. (Part 2 of 4.)

```

46
47 // multipurpose bubble sort; parameter compare is a pointer to
48 // the comparison function that determines sorting order
49 void bubble(int work[], size_t size, int (*compare)(int a, int b))
50 {
51     void swap(int *element1Ptr, int *element2ptr); // prototype
52
53     // loop to control passes
54     for (unsigned int pass = 1; pass < size; ++pass) {
55
56         // loop to control number of comparisons per pass
57         for (size_t count = 0; count < size - 1; ++count) {
58
59             // if adjacent elements are out of order, swap them
60             if ((*compare)(work[count], work[count + 1])) {
61                 swap(&work[count], &work[count + 1]);
62             }
63         }
64     }
65 }
66

```

Fig. 7.26 | Multipurpose sorting program using function pointers. (Part 3 of 4.)

```
67 // swap values at memory locations to which element1Ptr and
68 // element2Ptr point
69 void swap(int *element1Ptr, int *element2Ptr)
70 {
71     int hold = *element1Ptr;
72     *element1Ptr = *element2Ptr;
73     *element2Ptr = hold;
74 }
75
76 // determine whether elements are out of order for an ascending
77 // order sort
78 int ascending(int a, int b)
79 {
80     return b < a; // should swap if b is less than a
81 }
82
83 // determine whether elements are out of order for a descending
84 // order sort
85 int descending(int a, int b)
86 {
87     return b > a; // should swap if b is greater than a
88 }
```

Fig. 7.26 | Multipurpose sorting program using function pointers. (Part 4 of 4.)


```
Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 1
```

```
Data items in original order
```

```
2    6    4    8    10   12   89   68   45   37
```

```
Data items in ascending order
```

```
2    4    6    8    10   12   37   45   68   89
```

```
Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 2
```

```
Data items in original order
```

```
2    6    4    8    10   12   89   68   45   37
```

```
Data items in descending order
```

```
89   68   45   37   12   10    8    6    4    2
```

Fig. 7.27 | The outputs of the bubble sort program in Fig. 7.26.

7.12 Pointers to Functions (Cont.)

Using Function Pointers to Create a Menu-Driven System

- A common use of **function pointers** is in **text-based menu-driven systems**.
- A user is prompted to select an option from a menu (possibly from 1 to 5) by typing the menu item's number.
- Each option is serviced by a different function.
- Pointers to each function are stored in an array of pointers to functions.

7.12 Pointers to Functions (Cont.)

- The user's choice is used as an index in the array, and the pointer in the array is used to call the function.
- Figure 7.28 provides a generic example of the mechanics of defining and **using an array of pointers to functions**.
- We define three functions—`function1`, `function2` and `function3`—that each take an integer argument and return nothing.
- We store pointers to these three functions in array `f`

```
1 // Fig. 7.28: fig07_28.c
2 // Demonstrating an array of pointers to functions.
3 #include <stdio.h>
4
5 // prototypes
6 void function1(int a);
7 void function2(int b);
8 void function3(int c);
9
10 int main(void)
11 {
12     // initialize array of 3 pointers to functions that each take an
13     // int argument and return void
14     void (*f[3])(int) = { function1, function2, function3 };
15
16     printf("%s", "Enter a number between 0 and 2, 3 to end: ");
17     size_t choice; // variable to hold user's choice
18     scanf("%u", &choice);
19 }
```

Fig. 7.28 | Demonstrating an array of pointers to functions. (Part I of 3.)

```

20 // process user's choice
21 while (choice >= 0 && choice < 3) {
22
23     // invoke function at location choice in array f and pass
24     // choice as an argument
25     (*f[choice])(choice);
26
27     printf("%s", "Enter a number between 0 and 2, 3 to end: ");
28     scanf("%u", &choice);
29 }
30
31 puts("Program execution completed.");
32 }
33
34 void function1(int a)
35 {
36     printf("You entered %d so function1 was called\n\n", a);
37 }
38
39 void function2(int b)
40 {
41     printf("You entered %d so function2 was called\n\n", b);
42 }
43

```

Fig. 7.28 | Demonstrating an array of pointers to functions. (Part 2 of 3.)

```
44 void function3(int c)
45 {
46     printf("You entered %d so function3 was called\n\n", c);
47 }
```

Enter a number between 0 and 2, 3 to end: 0
You entered 0 so function1 was called

Enter a number between 0 and 2, 3 to end: 1
You entered 1 so function2 was called

Enter a number between 0 and 2, 3 to end: 2
You entered 2 so function3 was called

Enter a number between 0 and 2, 3 to end: 3
Program execution completed.

Fig. 7.28 | Demonstrating an array of pointers to functions. (Part 3 of 3.)

Program Exercise

- 修改fig07_16程式
- 輸出為4手牌，每手5張牌!

Program exercise

- Modify the program in Fig. 7.16 so that the card-dealing function deals a five-card poker hand. Then write the following additional functions:
 - Determine whether the hand contains a pair.
 - Determine whether the hand contains two pairs.
 - Determine whether the hand contains three of a kind (e.g., three jacks).
 - Determine whether the hand contains four of a kind (e.g. four aces).